

PROJECT#5 Re-implement PROJECT#4, Using PWM To Control A Motor, to use FreeRTOS's task management.

INSTRUCTOR'S OBJECTIVES: At the end of this lab, the student will have gained some basic exposure to the functionality of an RTOS through replacing the task manager of their previous lab with that provided by FreeRTOS.

STUDENT OBJECTIVES:

Overall objective: To replace the Cyclic Executive task manager of the previous lab (PWM motor control) with the equivalent FreeRTOS task manager/APIs while having their application perform exactly as before.

To accomplish this, the student will

1. Replace the task manager and wrapping functions with FreeRTOS task management APIs
2. Remove any unnecessary pre-existing task management routines

Rubric (Design of embedded program): The code (1) is built upon the code of the previous lab; (2) operates identically to the previous lab; and, (3) has the previous cyclic executive task manager and task wrappers replaced with the appropriate FreeRTOS task management APIs (i.e. function calls).

Rubric (Demonstration of successful operation): When testing, the grader is able to (1) download, cross-compile and run your program; (2) vary the intensity of the speed of the motor by pressing the button; (3) pressing the button multiple times causes the motor speed to vary in steps from off to maximum, and then maximum to off; (4) the motor will have 5 steps of brightness from off [0] to maximum [5]. Note: If the student was unable to get the previous lab fully functional, they will be evaluated upon their ability to successfully duplicate that functionality while successfully replacing the task management with FreeRTOS.

OPPORTUNITY FOR EXTRA CREDIT: For extra credit, the student can demonstrate the successful use of either a semaphore or mutex to control the sharing of the state of the button. (The button state has to be shared by the button detection task and the task that changes the motor speed [by changing the PWM duty cycle].)

Grading Rubric:

Subject	Sub-category	+1	+1
Documentation (max. 4)	Basic	(basic) understandable variable and constant names	(basic) no magic numbers
	Good	(good) professional, comprehensive and concise	(good) of function purpose, inputs and outputs
Code (max 5)			
	Readability	(readability) understandably structured / organized	(readability) uses understandable variable and constant names
			(readability) uses functions to clarify where appropriate
		(Extensibility) structured to facilitate possible future extension and expansion	(Debugging) structured to facilitate debugging
Execution (max 9)			
	Button Push Handling	Debounces the button state	Reads / saves the state of the push button
			Works async
	Motor Driver	Intensity demonstrated to change	setups up and uses a PWM digital pin
			Works async
	Operation Motor / LED	Intensity changes with each button push appropriately and asynchronously	intensity changes with each push of the button
	Total operation		All other mechanisms still work